

[College Logo]

Industrial Internship Report

on

BANKING INFORMATION SYSTEM

Prepared by

Marripelly Vikas

Python Internship & Project

Upskill Campus (USC) / UniConverge Technologies Pvt Ltd (UCT)

GitHub Repository

<https://github.com/marripellyvikas20-a11y/upskillcampus>

Code: BankingInformationSystem.py

Report: BankingInformationSystem_Vikas_USC_UCT.pdf

Executive Summary

This report presents the work completed during the Industrial Internship provided through Upskill Campus and The IoT Academy in collaboration with UniConverge Technologies Pvt Ltd (UCT). The project assigned for the Python internship was Banking Information System.

The project is a console-based Python application that demonstrates account creation, PIN-based authentication, deposit, withdrawal, fund transfer, balance inquiry, account details, and transaction history. It uses object-oriented programming, input validation, and error handling.

The internship provided practical exposure to problem solving, software design, implementation, testing, documentation, and GitHub-based submission.

TABLE OF CONTENTS

1 Preface	3
2 Introduction	4
2.1 About UniConverge Technologies Pvt Ltd	4
2.2 About Upskill Campus	5
2.3 The IoT Academy	5
2.4 Objectives	5
2.5 Reference / Glossary	6
3 Problem Statement	7
4 Existing and Proposed Solution	8
4.1 Code Submission	8
4.2 Report Submission	8
5 Proposed Design / Model	9
5.1 High Level Diagram	9
5.2 Low Level Diagram	9
5.3 Interfaces	9
6 Performance Test	10
6.1 Test Cases	10
6.2 Test Procedure	10
6.3 Performance Outcome	10
7 My Learnings	11
8 Future Work Scope	12

1 Preface

The Industrial Internship provided an opportunity to understand how programming concepts can be applied to practical problems. The project work involved understanding the problem statement, designing a solution, implementing the application, testing the main functions, and preparing technical documentation.

The Banking Information System was selected as a practical application because banking operations require structured account management, authentication, transaction processing, validation, and reliable handling of user input.

The internship offered exposure to an industry-oriented development process. The work included independent implementation, functional testing, report preparation, and GitHub submission.

Learning and overall experience: The project strengthened Python programming, object-oriented design, exception handling, transaction logic, testing, and documentation skills.

Acknowledgement: I express my gratitude to Upskill Campus, The IoT Academy, UniConverge Technologies Pvt Ltd, mentors, coordinators, and everyone who supported the internship and project completion.

Message to juniors and peers: Practical projects help convert theoretical knowledge into usable skills. Systematic testing and clear documentation are equally important parts of software development.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

UniConverge Technologies Pvt Ltd (UCT) is a technology organization working in digital transformation and industrial technology solutions. The internship material describes work involving Internet of Things (IoT), cyber security, cloud computing, machine learning, communication technologies, Java full stack, Python, and front-end technologies.

The internship ecosystem associated with UCT includes practical industrial applications such as IoT platforms, smart factory solutions, LoRaWAN-based solutions, industrial monitoring, smart metering, and predictive maintenance.

IoT Platform: The internship material describes an IoT platform for rapid application deployment with dashboards, analytics and reporting, alerts and notifications, third-party integration, and rule-engine support.

Smart Factory Platform: The material describes production and asset monitoring, OEE, predictive maintenance, scalable architecture, and SaaS-based solutions.

Industrial Solutions: The material references applications in agriculture technology, smart cities, industrial monitoring, smart street lighting, and smart water, gas, and electricity metering.

Predictive Maintenance: Industrial machine health monitoring and predictive-maintenance solutions use embedded systems, industrial IoT, and machine-learning technologies.

2.2 About Upskill Campus (USC)

Upskill Campus, along with The IoT Academy and in association with UniConverge Technologies, facilitates the execution of the internship process. The program provides learners with practical exposure, project experience, and opportunities to apply technical knowledge to industry-oriented assignments.

2.3 The IoT Academy

The IoT Academy is the EdTech division associated with UCT and supports technology-focused learning and certification programs. Within the internship ecosystem, it supports structured learning and practical project exposure.

2.4 Objectives of this Internship Program

- Gain practical experience of working on an industry-oriented project.
- Solve a real-world style problem using programming concepts.
- Improve understanding of Python and software development practices.
- Strengthen problem-solving, testing, documentation, and communication skills.
- Improve readiness for future software development opportunities.

2.5 Reference

- [1] Internship project instructions and problem statement.
[2] Python documentation and standard library concepts used during implementation.
[3] GitHub repository used for project code and report submission.

2.6 Glossary

Term	Meaning
Python	Programming language used to implement the project.
OOP	Object-Oriented Programming based on classes and objects.
PIN	Personal Identification Number used for authentication.
Transaction	Banking operation such as deposit, withdrawal, or transfer.
Validation	Checking whether input satisfies required rules.
GitHub	Platform used to host and share the project repository.

3 Problem Statement

The problem is to develop a simple Banking Information System that can manage customer accounts and perform essential banking operations through a user-friendly console interface.

The application should allow account creation, authentication, balance inquiry, deposit, withdrawal, fund transfer, account details, and transaction-history operations. The system must validate input and prevent invalid financial operations such as negative transactions or withdrawals beyond available balance.

Functional requirements:

- Create a unique account with customer information.
- Authenticate using account number and PIN.
- Deposit money into an authenticated account.
- Withdraw money when sufficient balance is available.
- Transfer funds between valid accounts.
- Display balance, account details, and transaction history.
- Provide a menu-driven interface with input validation.

4 Existing and Proposed Solution

Traditional manual records or very basic console programs may lack structured account management, authentication, transaction records, and validation. Manual processes can make balance maintenance and transaction tracing difficult.

The proposed solution is a Python console application organized using classes and data structures. Account objects maintain customer information, balance, PIN, and transaction history. The banking system manages accounts and routes menu operations.

The proposed solution adds automated transaction processing, authentication, validation, transaction history, and separation between account data and banking operations.

4.1 Code Submission (GitHub Link)

<https://github.com/marripellyvikas20-a11y/upskillcampus/blob/main/BankingInformationSystem.py>

4.2 Report Submission (GitHub Link)

https://github.com/marripellyvikas20-a11y/upskillcampus/blob/main/BankingInformationSystem_Vikas_USC_UCT.pdf

5 Proposed Design / Model

The system follows a simple flow: user input is received through the console, the banking system validates and routes the requested operation, account objects maintain account state, and transaction records store successful activity.

5.1 High Level Diagram

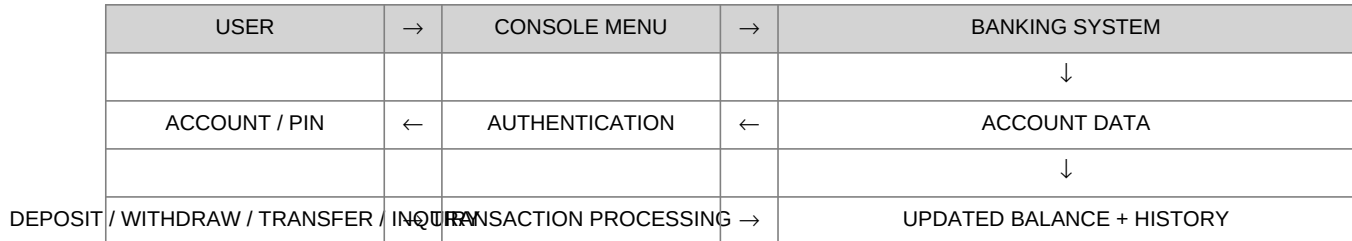


Figure 1: HIGH LEVEL FLOW OF THE BANKING INFORMATION SYSTEM

5.2 Low Level Diagram

BankingInformationSystem: manages accounts and provides create-account, login, deposit, withdraw, transfer, inquiry, details, and history operations.

Account: stores account number, customer name, phone number, PIN, balance, and transaction list.

Transaction: records transaction type, amount, timestamp, and description.

5.3 Interfaces

The project uses a command-line interface. The main menu provides numbered options and accepts input through standard Python console functions. Inputs are validated before operations are executed.

6 Performance Test

Performance testing for this console prototype focuses on functional correctness, input validation, transaction consistency, and response behavior rather than production-scale banking throughput.

6.1 Test Plan / Test Cases

ID	Test Case	Expected Result
TC01	Create account with valid details	Account created with unique number.
TC02	Enter invalid PIN	Authentication fails.
TC03	Deposit positive amount	Balance increases correctly.
TC04	Deposit zero/negative amount	Transaction rejected.
TC05	Withdraw within balance	Balance decreases correctly.
TC06	Withdraw above balance	Transaction rejected.
TC07	Transfer to valid account	Both balances update correctly.
TC08	Transfer to invalid account	Transfer rejected.
TC09	View transaction history	Recorded transactions displayed.
TC10	View account details	Correct details displayed.

6.2 Test Procedure

The application is started in a Python 3.x environment. The tester creates one or more accounts, authenticates into an account, and executes each banking operation from the menu. Both valid and invalid inputs are supplied to verify the validation logic.

For transaction tests, the balance before the operation is compared with the balance after the operation. Transfer tests compare both sender and receiver balances. Transaction-history output is checked to confirm that successful operations are recorded.

6.3 Performance Outcome

The implemented prototype supports the defined functional scenarios. Account creation, authentication, deposits, withdrawals, transfers, balance inquiry, account details, and transaction history are handled through the console workflow. Invalid amounts, invalid accounts, incorrect authentication, and insufficient funds are rejected.

The project is an internship prototype rather than a production banking platform. Production deployment would require persistent database storage, encryption, secure authentication, authorization, audit logging, concurrency control, and applicable security controls.

7 My Learnings

The internship provided practical experience in translating a problem statement into an implementable software solution. The Banking Information System strengthened the following skills:

- Python programming and object-oriented design.
- Classes, objects, lists, dictionaries, and data structures.
- Input validation and exception handling.
- Designing transaction workflows and maintaining consistent balances.
- Testing valid and invalid user scenarios.
- Technical documentation and report preparation.
- Using GitHub for code and report submission.
- Understanding how requirements influence system design.

These skills are useful for future software development because they combine programming knowledge with practical development, testing, documentation, and version-control practices.

8 Future Work Scope

The current application is a console-based prototype. Future versions can be extended with:

- MySQL or PostgreSQL database storage for persistent accounts.
- Secure PIN/password hashing.
- Graphical or web-based user interface.
- Administrator and customer role-based access.
- Interest calculation, loans, and account statements.
- Multi-factor authentication.
- Detailed audit logs and reporting.
- Automated unit and integration tests.
- Secure cloud deployment.

These enhancements would move the prototype closer to an industry-grade banking application while preserving the core account and transaction-management concepts demonstrated in the internship project.

Final GitHub Repository

<https://github.com/marripellyvikas20-a11y/upskillcampus>